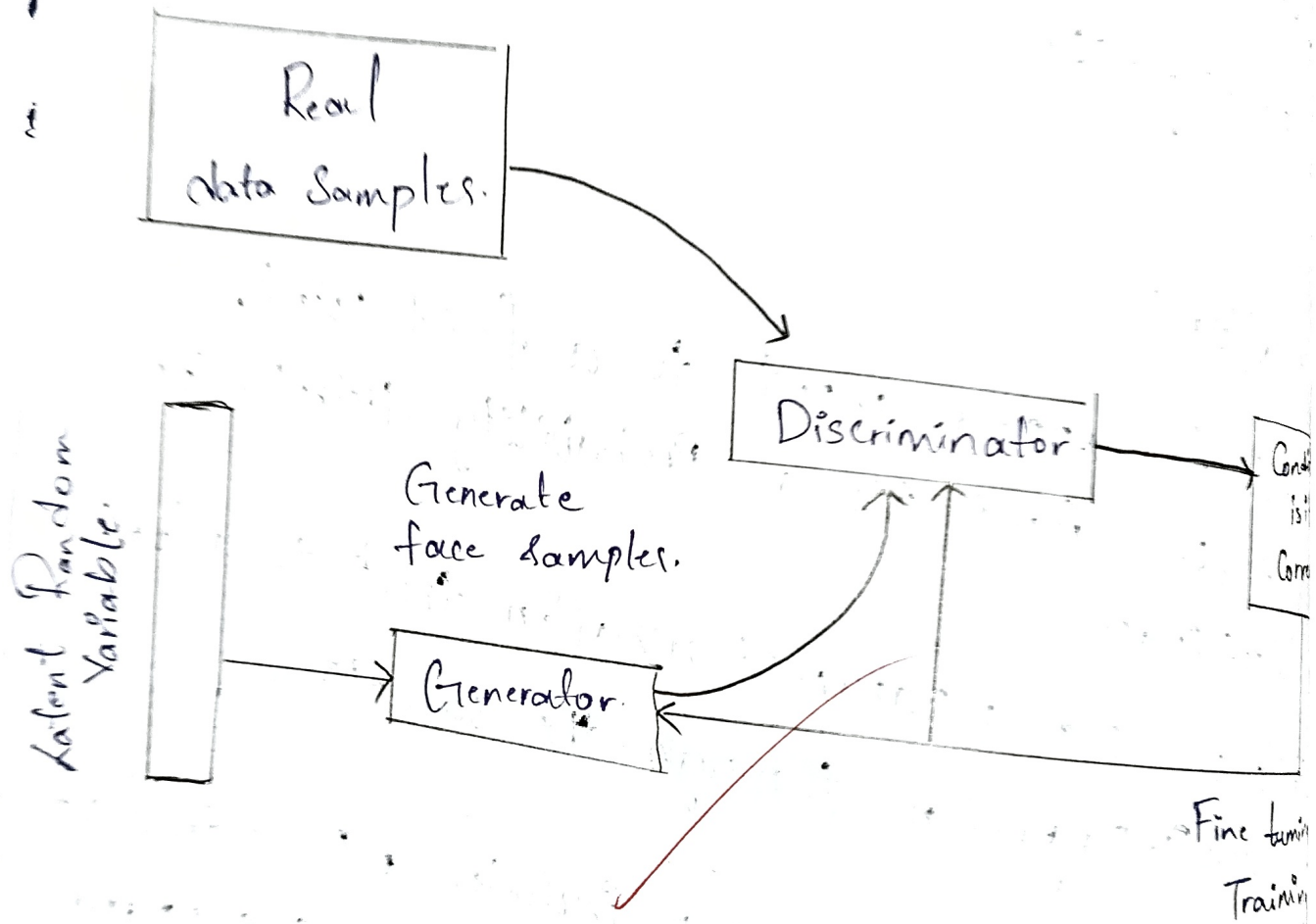


Q1

E

Architecture of GAN:



Ans.

Emp: 12 Implement GAN to generative images

Aim: To design and implement a Deep Convolutional generative Adversarial Network using Pytorch for generating realistic handwritten digit images

Objectives:

- Build a generator network using transposed convolutional layers to generate synthetic MNIST like images from random noise
- Build Discriminator network
- Train both network adversarially
- Optimize both networks using BCE loss and Adam optimizer

Pseudo Code

- Import Libraries
- Define generator Network

layers:

Conv Transpose 2d → Batch Norm → ReLU

Conv Transpose 2d → Batch Norm → ReLU

Conv Transpose 2d → Tanh

- Define Discriminator layers:

Conv 2d → Leaky ReLU

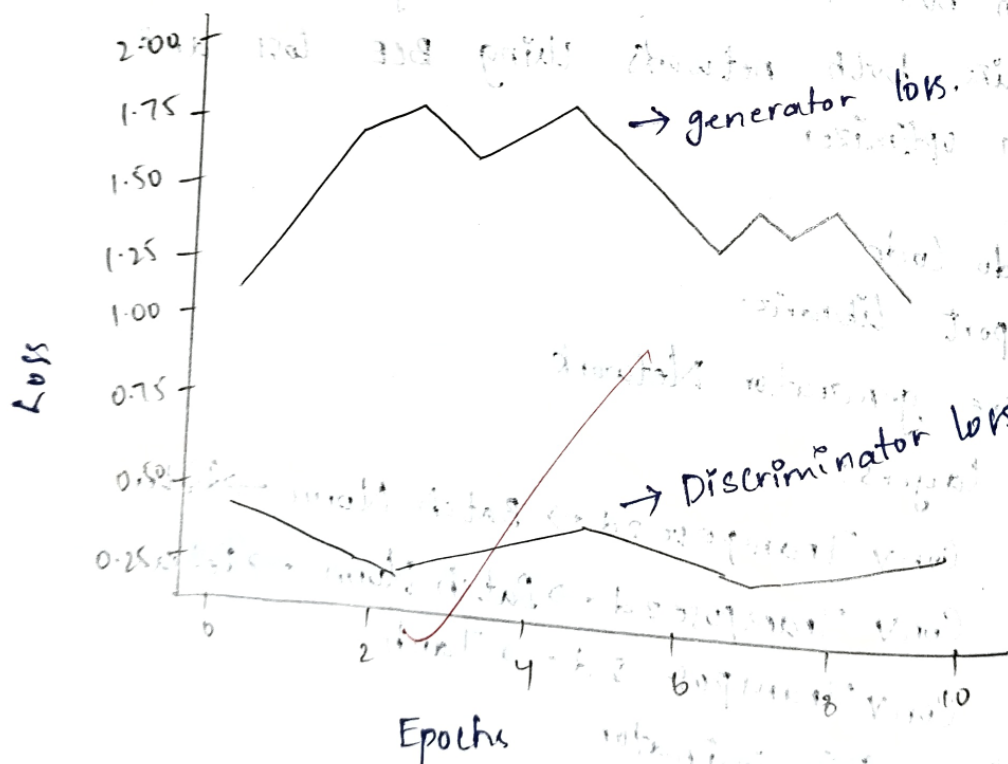
Conv 2d → Batch Norm → Leaky ReLU

Flatten → Linear → Sigmoid

Epoch (1/10) D Loss: 0.2675 G Loss: 1.3297
 Epoch (2/10) D Loss: 0.2787 G Loss: 1.8825
 Epoch (3/10) D Loss: 0.2546 G Loss: 2.0490

Epoch (6/10) D Loss: 0.3462 G Loss: 1.1048
 Epoch (7/10) D Loss: 0.3175 G Loss: 1.3139
 Epoch (10/10) D Loss: 0.3195 G Loss: 1.0122

Training Loss Curve Graph



- Initialize Comp
- Load Dataset
- Train generator
- optimizer
- Sample
- generate
- compute
- Backpropa
- Train Disc
- optimizer
- compute
- Compute

Backpropag
→ update R

Results!

generated

[Handwritten signature]

- Initialize Components
- Load Dataset
- Train generator
 - optimizer - G.Zero - grad()
 - Sample random noise (z)
 - generate fake-images = G(z)
 - compute g-loss = BCE (D(fake-images), real (labels))
 - Backpropagate and update generator

- Train Discriminator
 - optimizer - D.Zero - grad()
 - compute real-loss = BCE (D(real-images), real-labels)
 - compute fake-loss = BCE (D(fake-images), fake-labels)
 - Backpropagate and
- update Results

Results:-

Successfully implemented GAN and generated images

~~eff~~